SAECOIN

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Document Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Blockchain Type: Independent Layer-1
Consensus: Proof of Work (PoW)
Document Status: Planning
Development Status: Not Started

---

1. PURPOSE

This document defines the technical requirements, software architecture, development technologies, infrastructure, and implementation standards for SAECOIN.

SAECOIN will operate as an independent blockchain network rather than a token deployed on an existing blockchain.

The system will include:

- Blockchain Core
- Full Node
- Peer-to-Peer Network
- Consensus Engine
- Transaction Engine
- CPU Mining
- GPU Mining
- Wallet Applications
- Block Explorer
- Mining Pool Support
- Network Monitoring Tools

All technical decisions must prioritize security, reliability, decentralization, and long-term maintainability.

---

2. TARGET PLATFORMS

Initial development targets:

Platform| Purpose
Windows 11| Development, desktop wallet, node
Linux| Server, full node, mining

Android| Mobile wallet
Web| Block explorer

Future support may include macOS and additional platforms.

---

3. DEVELOPMENT COMPUTER SPECIFICATIONS

The primary development computer has the following specifications:

Component| Specification
Operating System| Windows 11
RAM| 16 GB
Development Environment| Windows-based
Primary IDE| Visual Studio Code / Rust tools
Version Control| Git
Source Repository| GitHub

Resource Management

Development tools must be selected with the available 16 GB RAM in mind.

Requirements:

- Avoid unnecessarily resource-heavy development environments.
- Build large components separately where possible.
- Avoid running multiple heavy services simultaneously.
- Use lightweight local testing environments.
- Optimize database and node memory usage.
- Allow development on a single computer before moving to dedicated servers.

The initial blockchain must be testable locally without requiring expensive server infrastructure.

---

4. PROGRAMMING LANGUAGES

The proposed technology stack is:

Component| Language
Blockchain Core| Rust
Full Node| Rust
Consensus Engine| Rust
Transaction Engine| Rust
P2P Network| Rust
CPU Miner| Rust

GPU Miner| Rust / GPU-specific backend
Wallet Core| Rust
Android Wallet| Kotlin
Android UI| Jetpack Compose
Windows Desktop Wallet| Rust + Tauri
Desktop UI| TypeScript
Block Explorer Frontend| React + TypeScript
Explorer Backend| Rust
Testing| Rust testing tools

Rust is selected for the blockchain core because of its memory safety, performance, and suitability for systems programming.

---

5. BLOCKCHAIN CORE REQUIREMENTS

The blockchain core must implement:

1. Block creation.
2. Block validation.
3. Transaction validation.
4. Proof-of-Work verification.
5. Difficulty adjustment.
6. Block reward calculation.
7. Halving mechanism.
8. UTXO management.
9. Chain selection and reorganization.
10. Maximum supply enforcement.

The core must operate independently without relying on another blockchain.

Consensus-critical rules must be deterministic across all nodes.

---

6. CONSENSUS REQUIREMENTS

Consensus Mechanism: Proof of Work.

Provisional parameters:

Parameter| Proposed Value
Maximum Supply| 21,000,000 SAC
Initial Block Reward| 50 SAC
Target Block Time| 10 minutes
Halving Interval| 210,000 blocks
Premine| 0

Decimal Precision| 8

These values are planning parameters and must be reviewed before implementation.

The final consensus specification must define:

- Mining algorithm.
- Block hash algorithm.
- Difficulty adjustment formula.
- Genesis block.
- Block validity rules.
- Subsidy calculation.
- Transaction verification rules.
- Chain selection rules.

The maximum supply must be enforced using deterministic integer arithmetic.

---

7. MINING REQUIREMENTS

SAECOIN should support:

CPU Mining

- CPU-based mining.
- Multi-core support.
- Mining status monitoring.
- Hashrate display.
- Mining reward tracking.

GPU Mining

- GPU mining support is a development objective.
- GPU-specific implementation must be evaluated against the final PoW algorithm.
- GPU backends must be benchmarked.
- Mining performance must be tested on supported hardware.

Mining Modes

- Solo mining.
- Mining pool support.
- Mining configuration.
- Mining statistics.
- Worker monitoring.

ASIC resistance is a design objective, not a guarantee.

---

8. FULL NODE REQUIREMENTS

A full node must:

- Download blockchain data.
- Verify blocks.
- Verify transactions.
- Maintain the UTXO set.
- Communicate with other nodes.
- Relay valid transactions.
- Relay valid blocks.
- Detect invalid blocks.
- Support chain synchronization.
- Recover after unexpected shutdowns.

The node must not blindly trust information received from peers.

---

9. PEER-TO-PEER NETWORK

The P2P network must support:

- Peer discovery.
- Peer connections.
- Block propagation.
- Transaction propagation.
- Network synchronization.
- Peer scoring.
- Misbehavior detection.
- Connection limits.
- Peer banning.

Network messages must be validated before processing.

The network protocol must be versioned to support future upgrades.

---

10. TRANSACTION ENGINE

SAECOIN will use the UTXO transaction model.

Requirements:

- Transaction creation.

- Transaction signing.
- Transaction verification.
- Input validation.
- Output validation.
- Double-spend prevention.
- Fee calculation.
- Mempool management.
- Transaction propagation.

All monetary values must use integer smallest units.

Floating-point arithmetic must never be used for consensus monetary calculations.

---

11. DATABASE REQUIREMENTS

The initial database technology under consideration is RocksDB.

It must support:

- Block storage.
- Block indexes.
- Transaction indexes.
- UTXO storage.
- Chain metadata.
- Undo data for chain reorganizations.

Database implementation must provide safe and atomic updates where required.

The final storage engine will be selected after performance and reliability testing.

Explorer indexes and other non-consensus data should remain separate from consensus-critical blockchain state.

---

12. WALLET REQUIREMENTS

Android Wallet

Technology:

- Kotlin
- Jetpack Compose

Features:

- Create wallet.
- Restore wallet.
- Receive SAC.
- Send SAC.
- QR code support.
- Transaction history.
- Balance display.
- Network status.
- Address management.

Windows Wallet

Technology:

- Rust
- Tauri
- TypeScript

Features:

- Wallet creation.
- Wallet restoration.
- Send and receive.
- Transaction history.
- Address management.
- Node connection.
- Network information.

Wallet Security

- Private keys must never be transmitted to remote servers.
- Seed phrases must not be stored in plaintext.
- Wallet secrets must be encrypted at rest.
- Wallet backup and restoration must be supported.
- Sensitive information must not appear in application logs.

---

13. BLOCK EXPLORER

The explorer must provide:

- Latest blocks.
- Block details.
- Transaction details.
- Transaction status.
- Address information.
- Network hashrate.

- Mining difficulty.
- Total circulating supply.
- Network statistics.

The explorer must read validated blockchain information.

Explorer indexes must not become the source of consensus truth.

---

14. MINING POOL REQUIREMENTS

Mining pool support is planned.

Potential features:

- Pool registration.
- Worker management.
- Share verification.
- Hashrate statistics.
- Reward accounting.
- Payout history.

Pool software must independently verify submitted shares.

Pool accounting must remain separate from blockchain consensus.

---

15. SECURITY REQUIREMENTS

Security is a critical requirement.

The system must include:

- Strict block validation.
- Strict transaction validation.
- Double-spend protection.
- Secure private-key handling.
- Peer abuse protection.
- Input validation.
- Safe database updates.
- Protection against malformed network messages.
- Secure wallet backups.
- Automated security testing.

Cryptographic algorithms and serialization rules must be explicitly specified and reviewed before mainnet.

---

16. TESTING REQUIREMENTS

Testing must include:

Unit Testing

Individual functions and modules.

Integration Testing

Communication between blockchain components.

Consensus Testing

Block and transaction validity.

Mining Testing

CPU and GPU mining correctness.

Network Testing

Peer connections and synchronization.

Wallet Testing

Wallet creation, recovery, signing, and transactions.

Security Testing

Invalid blocks, malformed messages, and attack scenarios.

Performance Testing

Memory usage, CPU usage, storage performance, and synchronization speed.

---

17. DEVELOPMENT ENVIRONMENT

The initial development environment will be Windows 11.

Required tools:

- Rust toolchain.

- Cargo.
- Git.
- GitHub.
- Visual Studio Code or another Rust-compatible IDE.
- Android Studio.
- Kotlin.
- Node.js.
- Tauri development tools.
- Database development tools.

Linux testing environments should be introduced when necessary.

The project should remain buildable and testable locally.

---

18. SERVER REQUIREMENTS

During initial development:

- Run the node locally.
- Use local test networks.
- Avoid unnecessary cloud hosting costs.
- Test mining and wallet connectivity locally.

For public testnet and mainnet:

- Dedicated Linux servers may be used.
- Nodes should be distributed across different locations.
- Monitoring and backups should be implemented.
- Server resources should be scaled based on actual network demand.

A single server must not become a central dependency for the entire blockchain.

---

19. SOURCE CODE MANAGEMENT

Git and GitHub will be used.

Requirements:

- Separate modules.
- Clear directory structure.
- Version-controlled source code.
- Documentation for each major module.
- Consistent naming conventions.
- Code review before major changes.

- Automated build and test workflows.

Private keys, passwords, API secrets, and other sensitive credentials must never be committed to GitHub.

---

20. PROPOSED PROJECT STRUCTURE

```
saecoin/
│
├── blockchain/
│   ├── core/
│   ├── consensus/
│   ├── transactions/
│   ├── storage/
│   └── network/
│
├── node/
│
├── miner/
│   ├── cpu/
│   └── gpu/
│
├── wallet/
│   ├── core/
│   ├── android/
│   └── desktop/
│
├── explorer/
│   ├── backend/
│   └── frontend/
│
├── mining-pool/
│
├── tests/
│
├── docs/
│
└── README.md
```

This structure is provisional and may change as the architecture becomes more detailed.

---

21. PERFORMANCE REQUIREMENTS

The system should be designed to:

- Operate on modest development hardware.
- Support efficient block validation.
- Minimize unnecessary memory usage.
- Synchronize reliably.
- Handle growing blockchain storage.
- Support multiple peer connections.
- Keep wallet operations responsive.

Performance targets must be established through benchmarking rather than assumptions.

---

22. DEVELOPMENT PHASES

Phase 1 — Documentation

- PRD.
- Application Flow.
- UI Specification.
- TRD.
- Data Schema.
- Blockchain Architecture.
- Consensus Specification.
- Security Specification.
- API Specification.
- Testing Strategy.

Phase 2 — Blockchain Foundation

- Cryptographic utilities.
- Block structure.
- Transaction structure.
- Genesis block.
- UTXO model.
- Consensus implementation.

Phase 3 — Node Development

- Full node.
- Database.
- P2P networking.
- Synchronization.

Phase 4 — Mining

- CPU miner.
- GPU mining research and implementation.

- Mining validation.

Phase 5 — Wallets

- Wallet core.
- Android wallet.
- Windows wallet.

Phase 6 — Explorer

- Explorer backend.
- Explorer frontend.
- Network statistics.

Phase 7 — Testing

- Local test network.
- Multi-node testing.
- Security testing.
- Performance testing.

Phase 8 — Public Testnet

- Public testnet deployment.
- Community testing.
- Bug fixes.

Phase 9 — Mainnet Preparation

- Independent security review.
- Consensus review.
- Genesis configuration.
- Mainnet release preparation.

---

23. IMPORTANT ENGINEERING RULES

1. Do not begin mainnet development before consensus rules are finalized.
2. Do not use floating-point numbers for monetary calculations.
3. Do not store plaintext wallet secrets.
4. Do not trust unverified peer data.
5. Keep consensus logic deterministic.
6. Separate consensus-critical data from operational data.
7. Test every major module independently.
8. Document protocol changes.
9. Maintain backward compatibility where required.
10. Do not promise ASIC resistance without evidence.

11. Do not assume GPU mining performance before benchmarking.
12. Do not launch mainnet without extensive testing and security review.

---

24. FINAL TECHNICAL DIRECTION

SAECOIN will be developed as an independent Proof-of-Work Layer-1 blockchain.

The proposed core technology is Rust.

The wallet ecosystem will include Android and Windows applications.

The project will initially be developed and tested on a Windows 11 computer with 16 GB RAM.

Linux infrastructure will be introduced when public network testing requires it.

All protocol parameters remain provisional until the dedicated consensus and architecture documents are finalized.

END OF TRD.md